

Expo Router

Navigation file-based pour applications mobiles

Expo Router	File-based Routing	Layouts	Navigation
Deep Links		Auth Gates	

Parcours de formation — Développement d'Applications Mobiles

Djiguitech · Social Seller · Tous niveaux

■ Compétences visées

- ✓ Comprendre le paradigme du routing basé sur les fichiers et ses avantages
- ✓ Structurer une application avec des layouts imbriqués et des groupes de routes
- ✓ Naviguer programmatiquement avec useRouter et gérer l'historique
- ✓ Passer des paramètres entre écrans avec useLocalSearchParams
- ✓ Protéger des routes avec des guards d'authentification
- ✓ Configurer les deep links et les URL schemes pour le mobile

Table des matières

Chapitre 1 — Le paradigme file-based routing

- 1.1 Routing impératif vs déclaratif
- 1.2 Convention de nommage des fichiers
- 1.3 Segments dynamiques
- 1.4 Routes spéciales

Chapitre 2 — Layouts et groupes

- 2.1 _layout.tsx — le layout racine
- 2.2 Groupes de routes (parenthèses)
- 2.3 Tabs et Stack imbriqués
- 2.4 Layouts partagés

Chapitre 3 — Navigation et paramètres

- 3.1 Link et useRouter
- 3.2 useLocalSearchParams
- 3.3 Gestion de l'historique
- 3.4 Navigation conditionnelle

Chapitre 4 — Authentification et guards

- 4.1 Le pattern auth gate
- 4.2 Redirections automatiques
- 4.3 Routes protégées et publiques
- 4.4 Persistance de session

Chapitre 5 — Deep links et URL scheme

- 5.1 Configuration du scheme
- 5.2 Universal links (iOS) et App Links (Android)
- 5.3 OAuth et retour d'URL

5.4 Debugging des liens

Le paradigme file-based routing

1.1 Routing impératif vs déclaratif

Avant Expo Router (v1 en 2022), la navigation en React Native était gérée avec React Navigation de façon impérative : vous définissiez chaque route et sa hiérarchie dans le code JavaScript. Expo Router adopte le paradigme file-based routing popularisé par Next.js : la structure de vos fichiers dans le dossier `app/` définit automatiquement les routes de l'application. Un fichier `app/inbox.tsx` devient la route `/inbox`. Un fichier `app/orders/[id].tsx` devient la route `/orders/:id`.

Ce changement de paradigme apporte plusieurs avantages concrets : la structure de l'application est visible immédiatement en regardant l'arborescence des fichiers, les routes sont fortement typées (TypeScript connaît les paramètres disponibles), le deep linking est configuré automatiquement, et la cohabitation web/mobile est facilitée (même structure, différents rendus).

1.2 Convention de nommage des fichiers

Expo Router suit des conventions précises pour nommer les fichiers. Comprendre ces conventions est essentiel pour structurer correctement une application.

Conventions de nommage Expo Router

Fichier	Route générée	Description
<code>app/index.tsx</code>	<code>/</code>	Page d'accueil
<code>app/inbox.tsx</code>	<code>/inbox</code>	Route statique
<code>app/orders/index.tsx</code>	<code>/orders</code>	Index d'un sous-dossier
<code>app/orders/[id].tsx</code>	<code>/orders/:id</code>	Segment dynamique
<code>app/orders/[...rest].tsx</code>	<code>/orders/*</code>	Catch-all (multi-segments)
<code>app/_layout.tsx</code>	<code>(layout)</code>	Layout racine (non-navigable)
<code>app/(tabs)/_layout.tsx</code>	<code>(group)</code>	Groupe sans impact sur l'URL
<code>app/+not-found.tsx</code>	<code>*</code>	Page 404 custom

Architecture type d'un projet Social Seller

```
# Structure typique d'une application Expo Router
app/
  _layout.tsx # Layout racine (providers, auth gate)
  index.tsx # Redirect vers (tabs) ou (auth)
  (auth)/
```

```
_layout.tsx # Layout pour les écrans non-connectés
login.tsx # /login
register.tsx # /register
(tabs)/
_layout.tsx # Layout avec barre d'onglets
inbox.tsx # /inbox (onglet 1)
orders.tsx # /orders (onglet 2)
stats.tsx # /stats (onglet 3)
conversation/
[id].tsx # /conversation/:id
orders/
[id].tsx # /orders/:id
```

1.3 Segments dynamiques

Un segment entre crochets [id] crée une route dynamique. La valeur du segment est accessible via le hook `useLocalSearchParams()`. Les segments dynamiques peuvent être imbriqués et combinés avec des sous-dossiers. La notation `[...rest]` capture plusieurs segments — utile pour des hiérarchies profondes.

Layouts et groupes

2.1 `_layout.tsx` — le layout racine

Chaque dossier peut contenir un `_layout.tsx` qui définit l'interface commune à toutes les routes du dossier. Le layout reçoit les routes enfants via le composant Slot (contenu quelconque) ou Stack/Tabs (navigateurs spécifiques). Le layout racine `app/_layout.tsx` est le point d'entrée de toute l'application — c'est là qu'on place les providers globaux (contexte d'auth, thème, notifications).

```
// app/_layout.tsx – layout racine

import { Stack } from 'expo-router';
import { useEffect } from 'react';
import { supabase } from '@lib/supabase';
import { AuthProvider } from '@context/auth';

export default function RootLayout() {
  return (
    <AuthProvider>
    <Stack screenOptions={{ headerShown: false }}>
      <Stack.Screen name='(tabs)' />
      <Stack.Screen name='(auth)' />
      <Stack.Screen
        name='conversation/[id]'
        options={{ headerShown: true, title: 'Conversation' }}
      />
    </Stack>
  </AuthProvider>
);
}
```

2.2 Groupes de routes (parenthèses)

Un dossier dont le nom est entre parenthèses — comme `(tabs)` ou `(auth)` — est un groupe de routes. Ce dossier n'apparaît PAS dans l'URL. Il sert uniquement à organiser les fichiers et partager un layout commun. Par exemple, `app/(tabs)/inbox.tsx` génère la route `/inbox` (pas `/(tabs)/inbox`). C'est l'outil central pour structurer les parties connectée et non-connectée de l'application.

```
// app/(tabs)/_layout.tsx – barre d'onglets

import { Tabs } from 'expo-router';
import { Ionicons } from '@expo/vector-icons';

export default function TabsLayout() {
  return (
    <Tabs
      screenOptions={{
        tabBarActiveTintColor: '#4F46E5',
        tabBarInactiveTintColor: '#94A3B8',
        headerShown: false,
      }}
    >
      <Tabs.Screen
        name='inbox'
        options={{
          title: 'Inbox',
          tabBarIcon: ({ color }) => (
            <Ionicons name='chatbubbles' size={24} color={color} />
          ),
        }}
      />
      <Tabs.Screen
        name='orders'
        options={{ title: 'Commandes', tabBarIcon: ({ color }) => (
          <Ionicons name='cart' size={24} color={color} />
        )}}
      />
    </Tabs>
  );
}
```


Navigation et paramètres

3.1 Link et useRouter

Expo Router offre deux façons de naviguer. Le composant Link est déclaratif — comme une balise `<a>` en HTML, il rend un élément cliquable qui navigue vers une route. useRouter est impératif — il expose des méthodes `push`, `replace`, `back`, `dismiss` pour naviguer depuis le code (après une validation, un submit de formulaire, etc.).

```
import { Link, useRouter } from 'expo-router';

// Navigation déclarative avec Link
function OrderCard({ order }) {
  return (
    <Link href={`\orders/${order.id}`} asChild>
      <TouchableOpacity>
        <Text>{order.customer_name}</Text>
      </TouchableOpacity>
    </Link>
  );
}

// Navigation impérative avec useRouter
function LoginForm() {
  const router = useRouter();

  async function handleLogin() {
    const { error } = await supabase.auth.signInWithPassword({ ... });
    if (!error) {
      router.replace('/(tabs)/inbox'); // replace = pas de retour arrière
    }
  }
}
```

3.2 useLocalSearchParams

Le hook `useLocalSearchParams()` retourne les paramètres de la route courante. Pour une route `/orders/abc-123`, `params.id` vaut `'abc-123'`. Tous les paramètres sont des strings (ou arrays de strings pour les catch-all). Convertir explicitement vers le bon type si nécessaire.

```
// app/orders/[id].tsx

import { useLocalSearchParams } from 'expo-router';

export default function OrderDetail() {
  const { id } = useLocalSearchParams<{ id: string }>();
  // id est maintenant typé – TypeScript sait qu'il existe

  const [order, setOrder] = useState<Order | null>(null);

  useEffect(() => {
    if (!id) return;
    fetchOrder(id).then(setOrder);
  }, [id]);

  if (!order) return <LoadingScreen />;
  return <OrderDetailView order={order} />;
}
```

Authentification et guards

4.1 Le pattern auth gate

La protection des routes est l'un des besoins les plus critiques dans toute application. Expo Router recommande le pattern 'auth gate' : dans le layout racine, on observe l'état d'authentification et on redirige automatiquement vers la bonne partie de l'application. L'utilisateur connecté va vers (tabs), l'utilisateur non-connecté va vers (auth).

Auth gate avec redirection automatique

```
// app/_layout.tsx – auth gate complet

import { useEffect } from 'react';
import { useRouter, useSegments } from 'expo-router';
import { useAuth } from '@context/auth';

function AuthGate({ children }) {
  const { session, loading } = useAuth();
  const segments = useSegments();
  const router = useRouter();

  useEffect(() => {
    if (loading) return; // Attendre le chargement initial

    const inAuthGroup = segments[0] === '(auth)';

    if (!session && !inAuthGroup) {
      // Non-connecté hors de (auth) → rediriger vers login
      router.replace('/(auth)/login');
    } else if (session && inAuthGroup) {
      // Connecté dans (auth) → rediriger vers l'app
      router.replace('/(tabs)/inbox');
    }
  }, [session, loading, segments]);

  if (loading) return <SplashScreen />;
  return children;
}
```

Deep links et URL scheme

5.1 Configuration du scheme

Un deep link est une URL qui ouvre directement un écran spécifique de l'application. Sur mobile, les apps enregistrent un URL scheme custom (ex: socialseller://) ou des universal links (https://). Expo Router génère automatiquement le linking configuration depuis la structure de fichiers. Il suffit de définir le scheme dans app.json.

```
// app.json – configuration du scheme

{
  "expo": {
    "scheme": "socialseller",
    "name": "Social Seller",
    "ios": {
      "bundleIdentifier": "com.djiguitech.socialseller"
    },
    "android": {
      "package": "com.djiguitech.socialseller"
    }
  }
}

// URLs de deep link résultantes :
// socialseller:// → app/(tabs)/inbox
// socialseller://orders → app/(tabs)/orders
// socialseller://orders/123 → app/orders/[id] avec id='123'

// Utilisation pour OAuth (retour après autorisation)
await WebBrowser.openAuthSessionAsync(authUrl, 'socialseller://');
```

■ À retenir

- ✓ La structure des fichiers dans app/ définit directement les routes
- ✓ Les groupes (parenthèses) organisent sans modifier les URLs
- ✓ _layout.tsx est partagé entre toutes les routes du dossier
- ✓ L'auth gate dans le layout racine gère toutes les redirections d'auth

- ✓ `useLocalSearchParams()` pour lire les paramètres de route — tous des strings
- ✓ Le `schema` dans `app.json` active les deep links automatiquement

—■ Exercices

Exercice 1 — Structure de navigation

Concevoir et implémenter la structure de navigation d'une app de recettes.

- Groupes : (`auth`) pour login/register, (`app`) pour les écrans principaux
- Tabs : recettes, favoris, profil
- Route dynamique : `recette/[id]` avec chargement des données
- Auth gate : redirection automatique selon l'état de connexion

Exercice 2 — Navigation avec paramètres

Construire un écran de détail qui reçoit des paramètres.

- Route `app/recettes/[id].tsx` avec `useLocalSearchParams`
- Charger les données depuis l'API en fonction de l'id
- Bouton retour avec `router.back()`
- Passer des paramètres supplémentaires via la query string (`?mode=edit`)

Exercice 3 — Deep link OAuth

Configurer un deep link pour le retour OAuth.

- Configurer le `schema` dans `app.json`
- Ouvrir une URL OAuth avec `WebBrowser.openAuthSessionAsync`
- Récupérer le code d'autorisation depuis l'URL de retour
- Tester le deep link avec `adb shell am start` sur Android

Bibliographie & Ressources

Documentation

- docs.expo.dev/router/introduction — documentation Expo Router
- docs.expo.dev/router/advanced/router-settings — configuration avancée
- expo.github.io/router/docs — référence complète de l'API

Ressources

- Simon Grimm — tutoriels Expo Router (YouTube)
- notjust.dev — exemples de projets avec Expo Router